

# SEARCHING

## PROBLEM 8

### QUESTION:

| DS                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       | Session | Searching | Question Information | Level 1 | Challenge 8 |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------|-----------|----------------------|---------|-------------|
| <p><b>Problem Description:</b><br/>Ram's first job is a out of university is for a student's book publisher. They are currently publishing a line of "intelligent libs" books, and they need you to check their print-copy. Write a program that will take a sentence with placeholders in it, and substitute in for those placeholders words from the lists of example words the publisher has provided.</p> <p>Your goal will be to substitute the appropriate words into the sentence from the list of provided words TWICE (keeping track of the words you have used) where:</p> <p>[N] = nouns<br/>[AV] = adverbs<br/>[V] = verbs<br/>[AJ] = adjectives</p> <p><b>Input Format:</b><br/>Line one will always be the line with the story. The order of the Nouns, Adverbs, Verbs and Adjectives will always be the same. Input stops when it reaches the word END alone on a line. You are guaranteed to always have enough words to substitute into the sentence TWICE (not repeating any words). The number of words given under each section will be random.</p> <p><b>Output Format:</b><br/>Print the output in a separate lines contains, Always take the TOP:most UNUSED word from a group when substituting. Output the final sentences when all substitutions are done.</p> |         |           |                      |         |             |
| <p>▼ Logical Test Cases</p> <div><div><p>Test Case 1</p><p>INPUT (STDIN)</p><p>There once was a [AJ] [N] from [N] who [AV] [V] all day.</p><p>NOUNS<br/>squirrel<br/>London<br/>tree<br/>lettuce<br/>ADVERBS<br/>silently<br/>quickly</p></div><div><p>Test Case 2</p><p>INPUT (STDIN)</p><p>There once was a [AJ] [N] from [N] who [AV] [V] all day.</p><p>NOUNS<br/>singer<br/>chennai<br/>hotel<br/>chettinad<br/>ADVERBS<br/>silently<br/>quickly</p></div></div>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |         |           |                      |         |             |
| <div><div><p>happily<br/>VERBS<br/>skipped<br/>ran<br/>climbs<br/>ADJECTIVES<br/>delightful<br/>homely<br/>quain</p><p>EXPECTED OUTPUT</p><p>There once was a delightful squirrel from London who silently skipped all day.<br/>There once was a homely tree from lettuce who quickly ran all day.</p></div><div><p>happily<br/>sadly<br/>VERBS<br/>skipped<br/>reads<br/>ran<br/>sings<br/>climbs<br/>ADJECTIVES<br/>delightful<br/>homely<br/>surprisingly</p><p>EXPECTED OUTPUT</p><p>There once was a delightful singer from chennai who silently skipped all day.<br/>There once was a homely hotel from chettinad who quickly reads all day.</p></div></div>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |         |           |                      |         |             |
| <p>▼ Mandatory Test Cases</p> <div><div><p>Test Case 1</p><p>KEYWORD</p><p>int cl[CMDS]</p></div><div><p>Test Case 2</p><p>KEYWORD</p><p>char *lists[CMDS] [MAXWORDS];</p></div><div><p>Test Case 3</p><p>KEYWORD</p><p>char *tokens[TOKENS]={ "[N]", "[AV]", "[V]", "[AJ]" };</p></div><div><p>Test Case 4</p><p>KEYWORD</p><p>char *cmds[CMDS]={ "NOUNS", "ADVERBS", "VERBS", "ADJECTIVE S", "END" };</p></div></div>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |         |           |                      |         |             |
| <p>▼ Complexity Test Cases</p> <div><div><p>Test Case 1</p><p>CYCLOMATIC COMPLEXITY</p><p>3</p></div><div><p>Test Case 2</p><p>TOKEN COUNT</p><p>488</p></div><div><p>Test Case 3</p><p>NLOC</p><p>50</p></div></div>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |         |           |                      |         |             |

## SOURCE CODE:

```
#include <stdio.h>
#include <string.h>

#define NUM_CATEGORIES 4

char nouns[20][50], adverbs[20][50], verbs[20][50], adjectives[20][50];
int noun_cnt = 0, adv_cnt = 0, verb_cnt = 0, adj_cnt = 0;
int noun_idx = 0, adv_idx = 0, verb_idx = 0, adj_idx = 0;

void replace_first(char *source, const char *pattern, const char *replacement) {
    char *match = strstr(source, pattern);
    if (!match) return;
    char buffer[1000];
    int head_len = match - source;
    strncpy(buffer, source, head_len);
    buffer[head_len] = '\0';
    strcat(buffer, replacement);
    strcat(buffer, match + strlen(pattern));
    strcpy(source, buffer);
}

int main() {
    char base_story[500];
    if (!fgets(base_story, sizeof(base_story), stdin)) return 0;
    base_story[strcspn(base_story, "\r\n")] = '\0';

    char cur_line[100];
    int section = -1;
    while (fgets(cur_line, sizeof(cur_line), stdin)) {
        cur_line[strcspn(cur_line, "\r\n")] = '\0';
        if (strcmp(cur_line, "END") == 0) break;
        if (strcmp(cur_line, "NOUNS") == 0) section = 0;
        else if (strcmp(cur_line, "ADVERBS") == 0) section = 1;
        else if (strcmp(cur_line, "VERBS") == 0) section = 2;
        else if (strcmp(cur_line, "ADJECTIVES") == 0) section = 3;
        else {
            if (section == 0) strcpy(nouns[noun_cnt++], cur_line);
            else if (section == 1) strcpy(adverbs[adv_cnt++], cur_line);
            else if (section == 2) strcpy(verbs[verb_cnt++], cur_line);
            else if (section == 3) strcpy(adjectives[adj_cnt++], cur_line);
        }
    }

    for (int iter = 0; iter < 2; iter++) {
        char populated[1000];
        strcpy(populated, base_story);

        while (strstr(populated, "[AJ]")) replace_first(populated, "[AJ]", adjectives[adj_idx++]);
        while (strstr(populated, "[N]")) replace_first(populated, "[N]", nouns[noun_idx++]);
        while (strstr(populated, "[AV]")) replace_first(populated, "[AV]", adverbs[adv_idx++]);
        while (strstr(populated, "[V]")) replace_first(populated, "[V]", verbs[verb_idx++]);

        printf("%s\n", populated);
    }
    return 0;
}
```

## OUTPUT:

There once was a [AJ] [N] from [N] who [AV] [V] all day.

NOUNS

squirrel

London

tree

lettuce

ADVERBS

silently

quickly

happily

VERBS

skipped

ran

climbs

ADJECTIVES

delightful

homely

END

There once was a delightful squirrel from London who silently skipped all day.

There once was a homely tree from lettuce who quickly ran all day.

There once was a [AJ] [N] from [N] who [AV] [V] all day.

NOUNS

singer

chennai

hotel

chettinad

ADVERBS

silently

quickly

happily

sadly

VERBS

skipped

reads

ran

sings

climbs

ADJECTIVES

delightful

homely

surprisingly

END

There once was a delightful singer from chennai who silently skipped all day.

There once was a homely hotel from chettinad who quickly reads all day.